Walmart Sales Analysis Project

Objective

The goal of this project is to analyze Walmart's sales data and derive meaningful insights related to store performance and sales trends. The idea is to build a reliable data pipeline that transforms raw data into a structured format that can be easily used for analysis and reporting.

Abstract

Handling retail data at scale can be challenging, so this project uses a modern and scalable data stack to simplify the process.

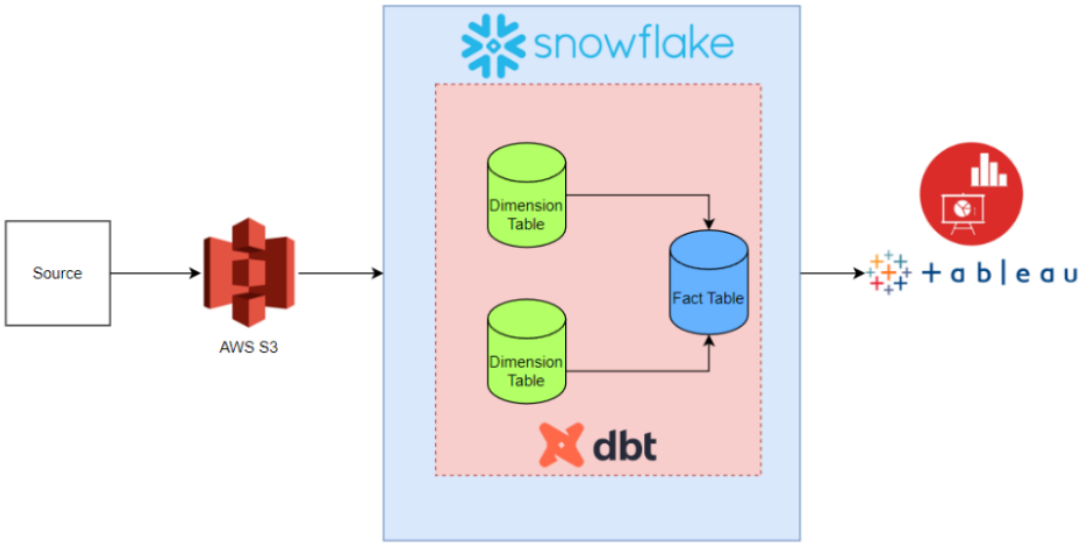
Snowflake is used as the cloud data warehouse to store and manage the data efficiently. Its scalability and performance make it well-suited for handling large datasets like Walmart's.

For transformations, dbt (Data Build Tool) is used. It allows us to build modular, version-controlled, and well-structured data pipelines while also providing lineage tracking.

For reporting, tools like Tableau or Python-based visualizations can be used to create dashboards and extract insights from the final data models.

Technology Used

- Snowflake (Database)
- dbt (Transformation Tool)
- Python (Data Handling & Reporting)
- Tableau / Any Reporting Tool



Source Data

The project uses three main datasets:

department.csv → Department-level sales data

fact.csv → Weekly sales and related metrics

stores.csv → Store-level attributes

Data Preparation

The following steps were performed before ingestion:

- Converted .xlsx files into .csv format
- Ensured correct headers and structure
- Verified that data is clean and consistent
- Stored all files in the local directory:

  C:\Users\BANSARI MODI\Documents\e2e_projects\Walmart\data

S3 Setup

An Amazon S3 bucket was created to store raw data files:

Bucket Name: walmart-dbt-project
Folder: data/

This acts as the **raw data layer** in the pipeline.

File Upload

Data was uploaded to S3 using a simple Python script with boto3.
The script reads all CSV files from the local folder and uploads them to the specified S3 bucket.

IAM Consideration

While uploading data, permission issues were encountered due to missing access rights.

The IAM user required permissions such as:

s3:PutObject

s3:GetObject

s3:ListBucket

After updating the IAM policy, the upload process was completed successfully.

Final S3 Structure

After ingestion, the S3 bucket contains:

s3://walmart-dbt-project/data/department.csv
s3://walmart-dbt-project/data/fact.csv
s3://walmart-dbt-project/data/stores.csv

Snowflake Setup

After uploading the data to S3, Snowflake was configured to securely access the data.

An external storage integration was created to connect Snowflake with the S3 bucket. This allows Snowflake to read data directly from S3 without manually copying files.

Key steps performed:

Created database: WALMART_DB

Created schemas: BRONZE, SILVER, GOLD, SNAPSHOTS

Created storage integration using IAM role

Configured external stage pointing to S3 path: s3://walmart-dbt-project/data/

Created CSV file format for parsing input files

An important step in this setup was configuring the IAM role trust relationship. Snowflake required permission to assume the AWS role, which was resolved by updating the IAM trust policy using the external ID and IAM user ARN provided by Snowflake.

Bronze Layer (Raw Data Ingestion)

The Bronze layer represents the raw ingestion of data from S3 into Snowflake.

In this project, dbt pre-hooks were used to execute COPY INTO commands, which load data from the Snowflake stage into Bronze tables.

Three raw tables were created:

- BRONZE.STORES
- BRONZE.DEPARTMENT
- BRONZE.FACT

Key characteristics of Bronze layer:

- Data is loaded as-is from source files
- Minimal transformation is applied
- Data types such as DATE and ISHOLIDAY are kept as VARCHAR to avoid ingestion errors
- Metadata columns are added:
    - INSERT_DTS
    - UPDATE_DTS
    - SOURCE_FILE_NAME
    - SOURCE_FILE_ROW_NUMBER

This layer acts as a reliable raw data source for downstream transformations.

Silver Layer (Data Cleaning & Transformation)

The Silver layer is responsible for cleaning and standardizing the raw data.

Three cleaned models were created:

- department_clean
- store_clean
- store_metrics_clean

Transformations performed:

department_clean

- Converted STORE and DEPT to integer
- Converted DATE from string to proper date format
- Converted WEEKLY_SALES to decimal
- Standardized ISHOLIDAY into boolean values

store_clean

- Converted STORE to integer
- Cleaned and trimmed TYPE
- Converted SIZE to integer

store_metrics_clean

- Converted DATE to proper date
- Cast numeric fields like temperature, fuel price, CPI, unemployment
- Standardized ISHOLIDAY

Key benefits of Silver layer:

- Ensures consistent data types
- Removes ambiguity from raw data
- Prepares data for analytical modeling

Gold Layer (Dimensional Modeling)

The Gold layer contains business-ready tables used for reporting and analysis.

1. Date Dimension (date_dim)

- Built from distinct dates in department_clean
- Generates a surrogate key: date_id
- Stores date and holiday information

2. Store Dimension (store_dim)

- Combines store and department information
- Contains:
    - store_id
    - dept_id
    - store_type
    - store_size
- Implements SCD1 logic (latest values overwrite previous ones)

Snapshot (SCD2 Implementation)

To track historical changes, dbt snapshots were used.

A snapshot named store_snapshots was created on top of store_dim.

Key configurations:

- Unique key: store_id + dept_id
- Strategy: check
- Columns tracked: store_type, store_size

This snapshot generates:

- dbt_valid_from
- dbt_valid_to

This enables SCD2 behavior, allowing us to track how store attributes change over time.

Fact Table (Final Analytical Table)

The final fact table fact_table is created in the Gold layer.

It is built by joining:

- store_snapshots (for SCD2 history)
- department_clean (sales data)
- store_metrics_clean (external metrics)
- date_dim (date key)

Features:

- Contains key measures like sales, fuel price, temperature, CPI, etc.
- Uses date_id for proper modeling
- Includes version columns (vrsn_start_date, vrsn_end_date)

This is the main table used for analysis and reporting.

dbt Testing

To ensure data quality, dbt tests were implemented.

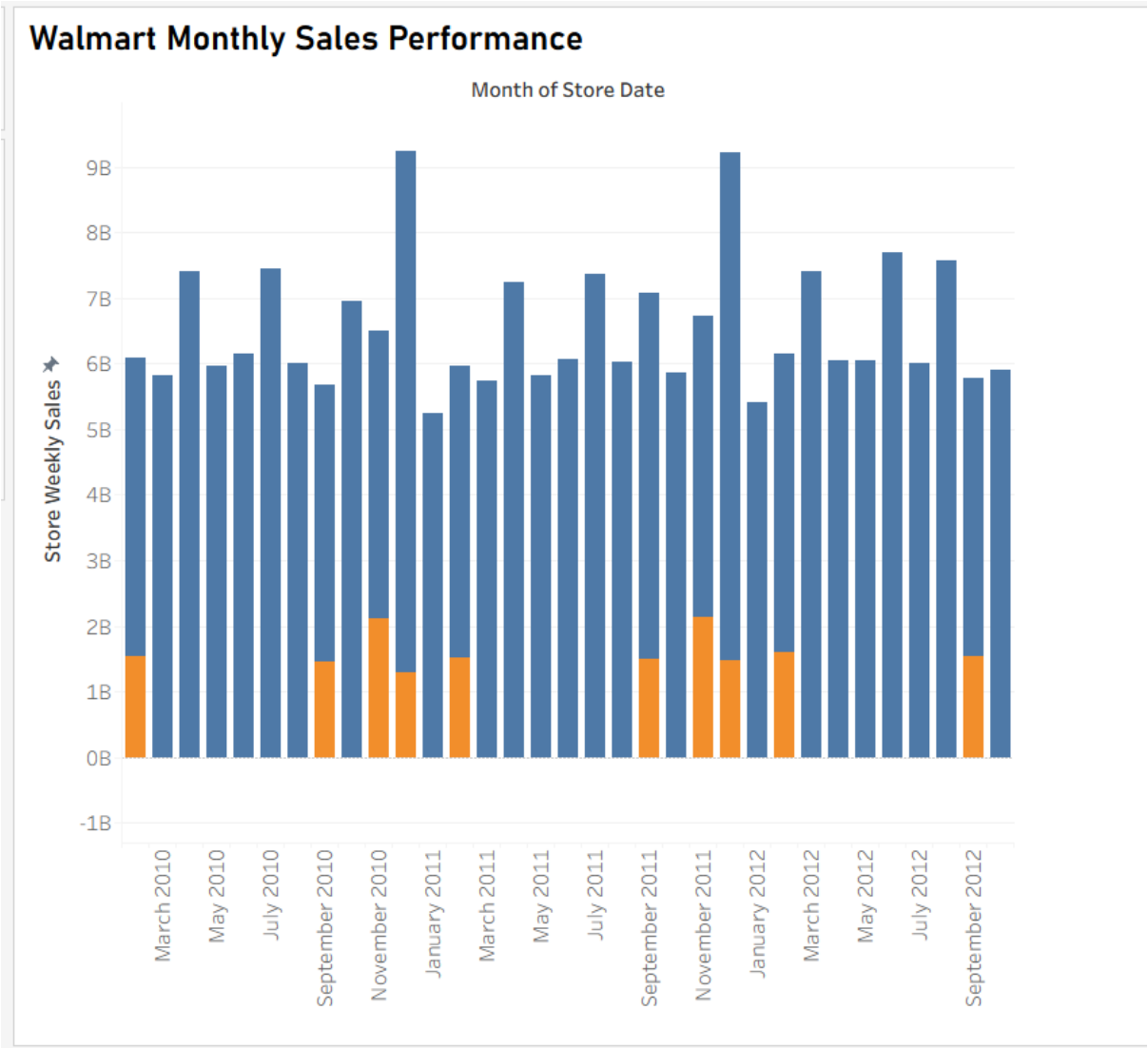
Tests added:

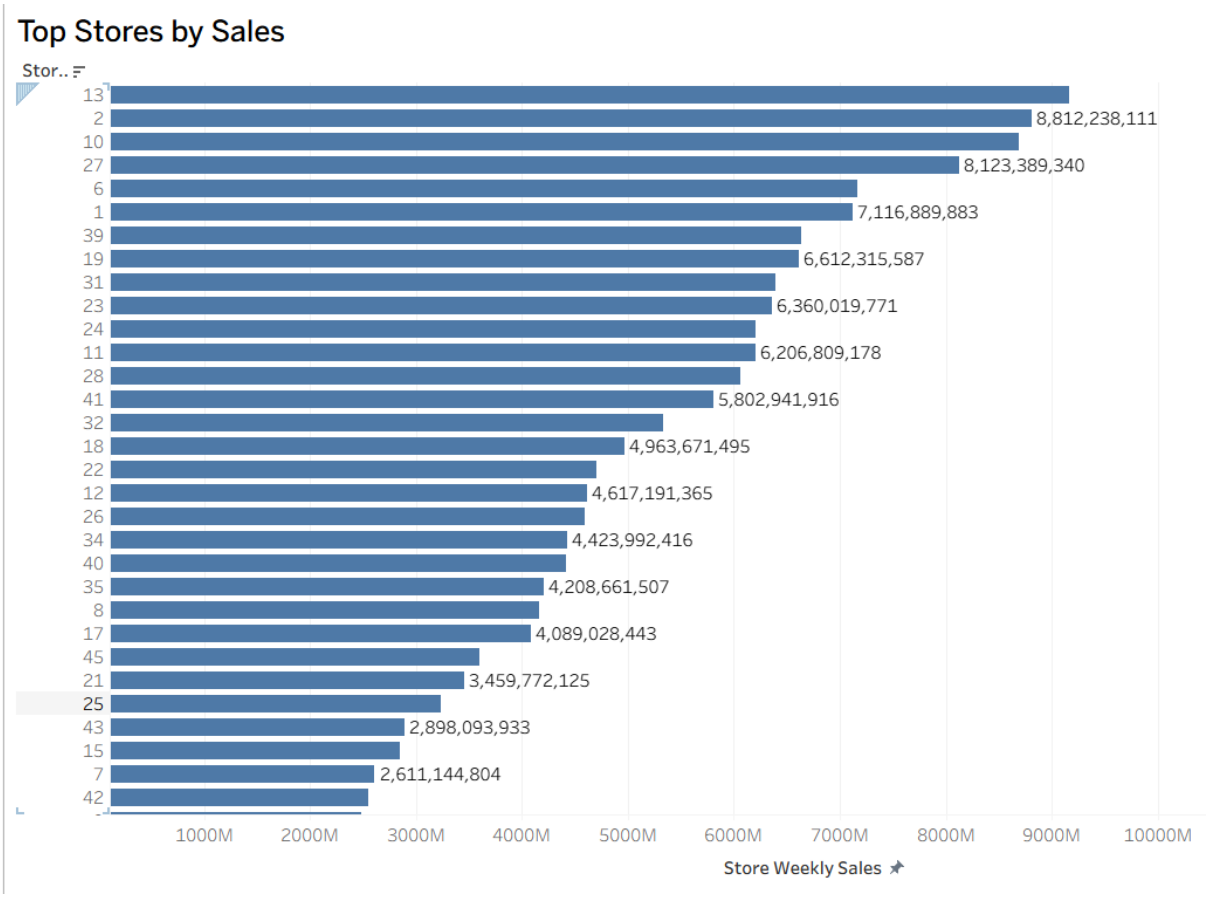
- not_null → ensures no missing values in key columns
- unique → ensures primary keys are unique
- relationships → ensures referential integrity between tables

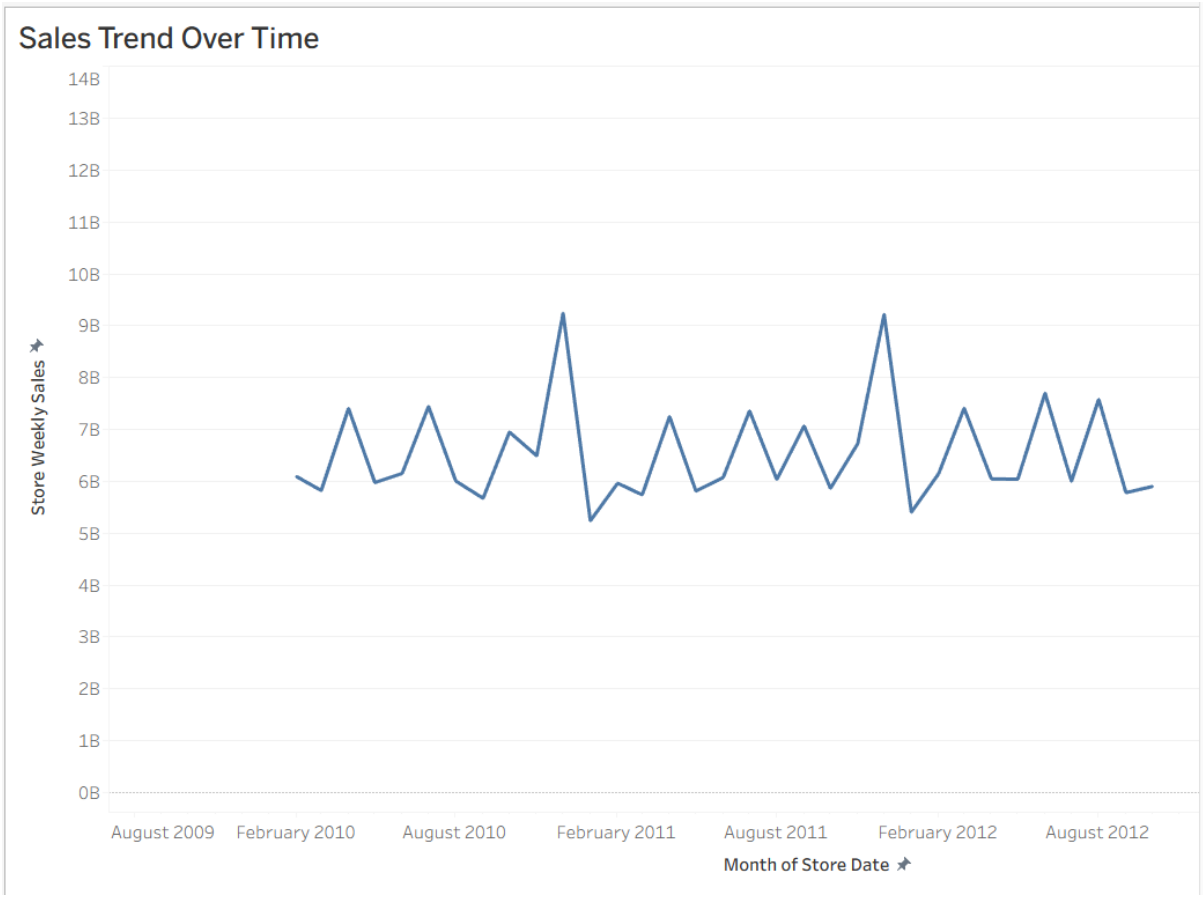
Example:

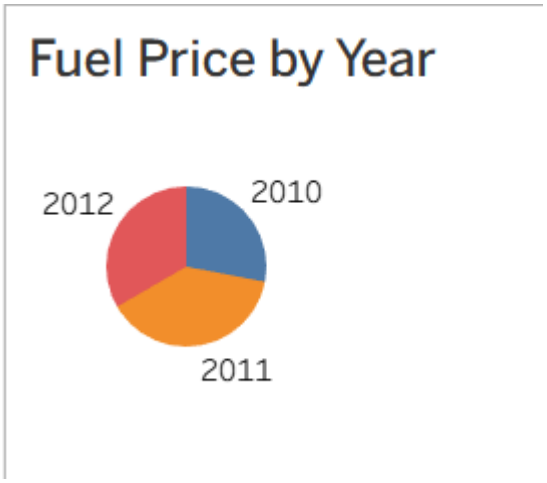
- date_dim.date_id → unique, not null
- fact_table.date_id → must exist in date_dim

All tests passed successfully, confirming that the data is clean, consistent, and reliable.

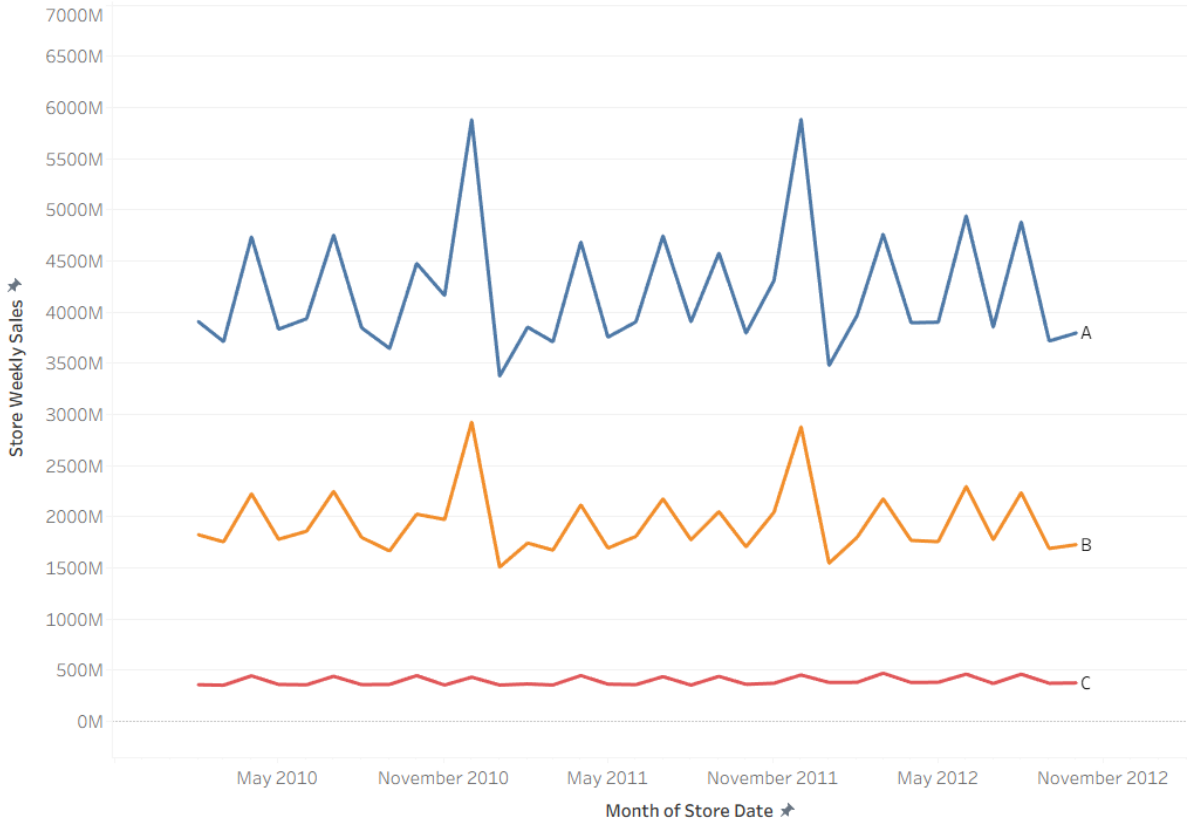

Walmart Monthly Sales Performance
Month of Store Date
Store Weekly Sales
9B
8B
7B
6B
5B
4B
3B
2B
1B
0B
-1B
March 2010
May 2010
July 2010
September 2010
November 2010
January 2011
March 2011
May 2011
July 2011
September 2011
November 2011
January 2012
March 2012
May 2012
July 2012
September 2012


Top Stores by Sales
Stor..
13
2
10
27
6
1
39
19
31
23
24
11
28
41
32
18
22
12
26
34
40
35
8
17
45
21
25
43
15
7
42
8,812,238,111
8,123,389,340
7,116,889,883
6,612,315,587
6,360,019,771
6,206,809,178
5,802,941,916
4,963,671,495
4,617,191,365
4,423,992,416
4,208,661,507
4,089,028,443
3,459,772,125
2,898,093,933
2,611,144,804
1000M
2000M
3000M
4000M
5000M
6000M
7000M
8000M
9000M
10000M
Store Weekly Sales

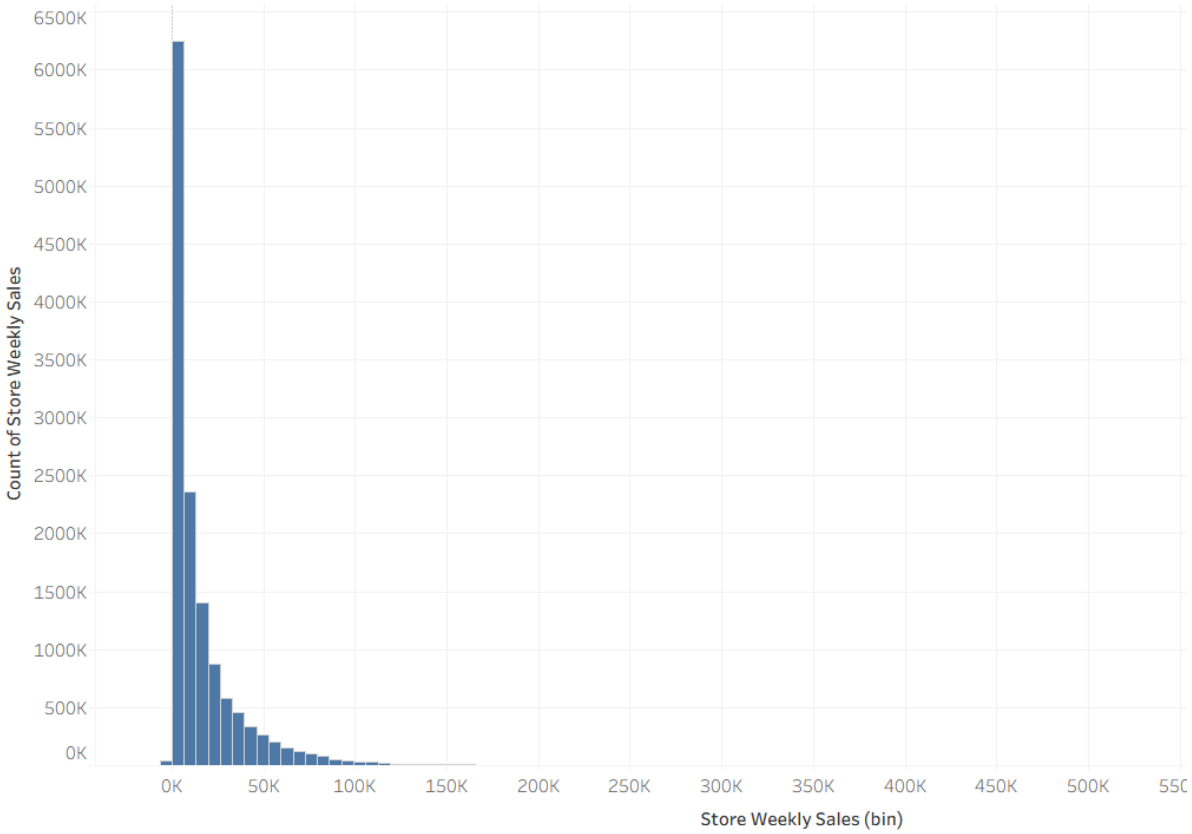

Sales Trend Over Time
14B
13B
12B
11B
10B
9B
8B
7B
6B
5B
4B
3B
2B
1B
0B
Store Weekly Sales
August 2009
February 2010
August 2010
February 2011
August 2011
February 2012
August 2012
Month of Store Date

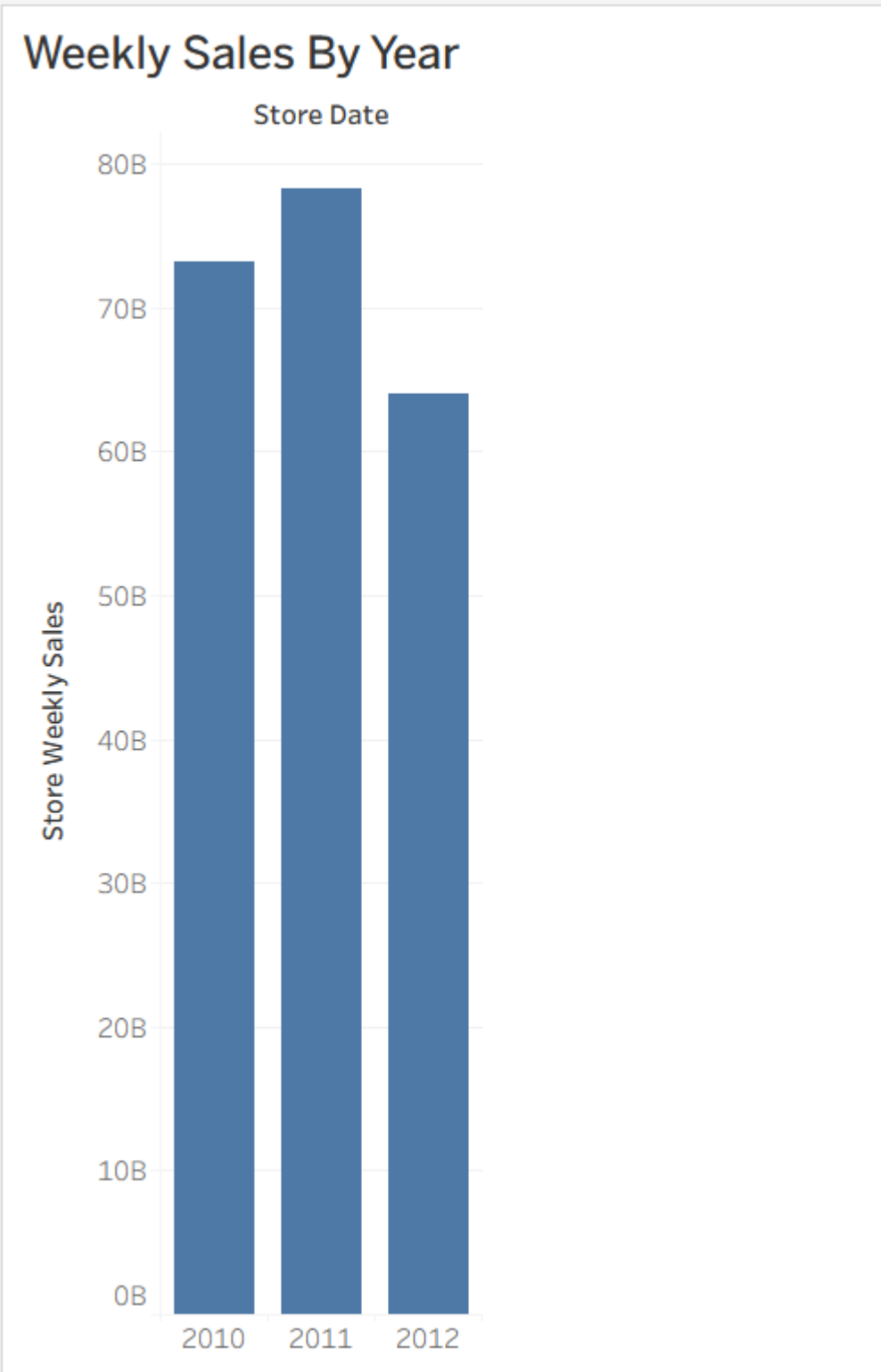

Fuel Price by Year
2012
2010
2011

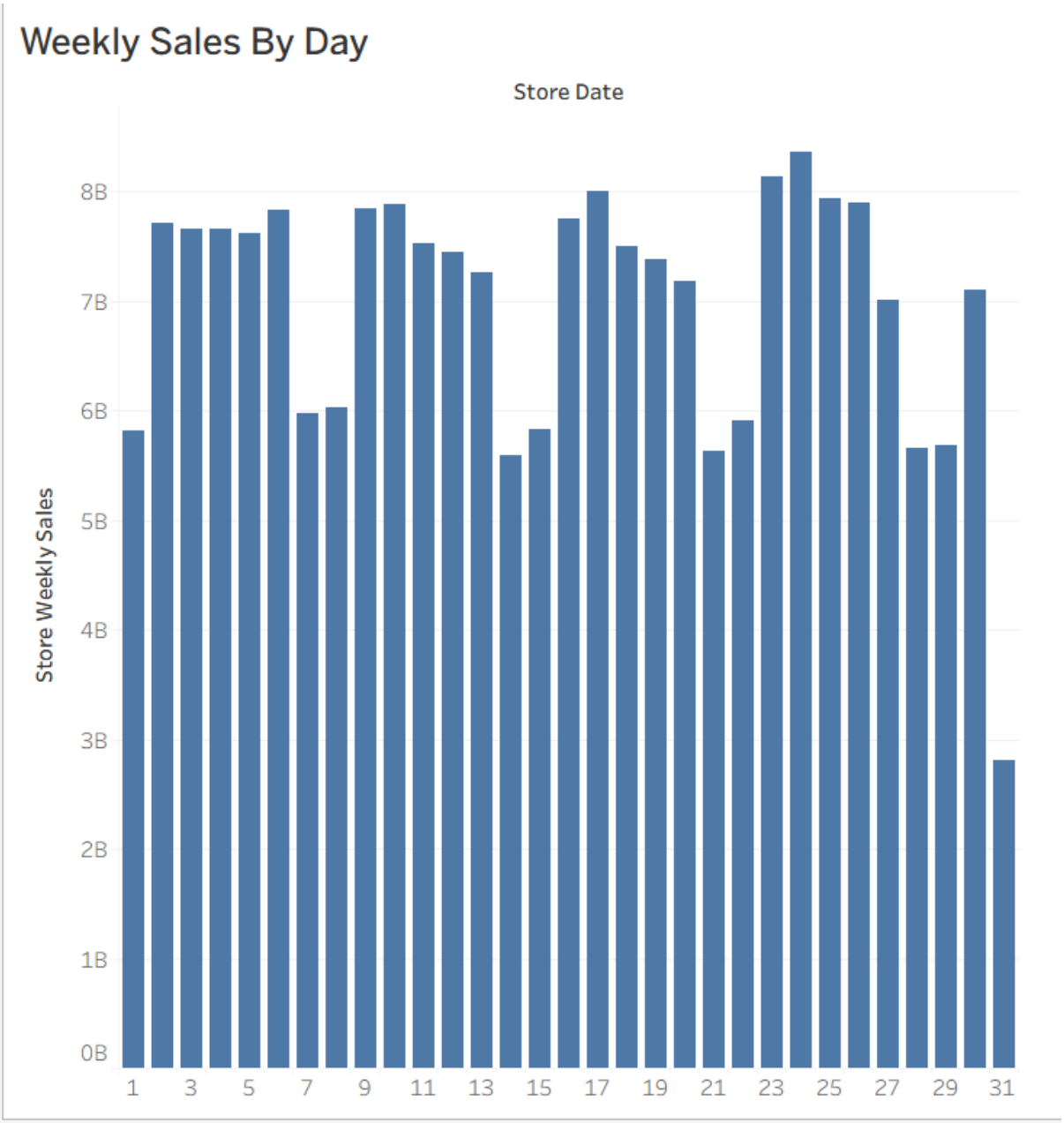
Weekly Sales by Store type and Month

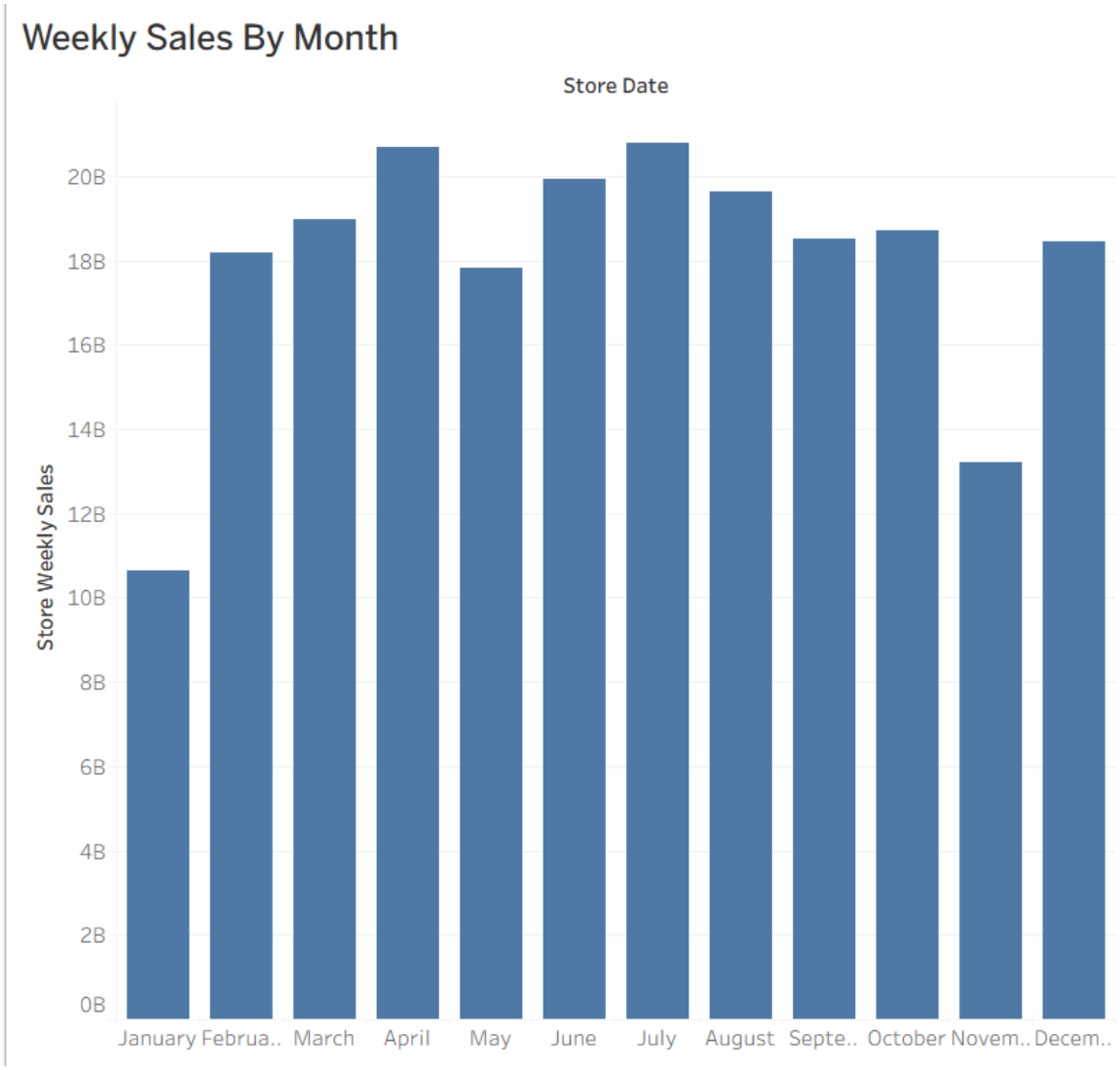


Sales Distribution




Weekly Sales By Year
Store Date
Store Weekly Sales
80B
70B
60B
50B
40B
30B
20B
10B
0B
2010
2011
2012


Weekly Sales By Day
Store Date
Store Weekly Sales
8B
7B
6B
5B
4B
3B
2B
1B
0B
1
3
5
7
9
11
13
15
17
19
21
23
25
27
29
31


Weekly Sales By Month
Store Date
Store Weekly Sales
20B
18B
16B
14B
12B
10B
8B
6B
4B
2B
0B
January
Februa..
March
April
May
June
July
August
Septe..
October
Novem..
Decem..